C++ 后端 / Linux 方向 面试八股高频考点整理

> 来源：综合牛客面经、卡码笔记、小林 coding、CSDN/B 站等 2024–2025 年公开的 C++ 后台面经（字节、腾讯、华为、小米、虎牙、英伟达等）中的高频题。 用法：每题先背“一句话答案”，再能展开 2–3 个要点和一个追问。★ 越多越高频。

一、C++ 对象与内存

1. RAII 是什么？★★★

- 资源获取即初始化：构造时获取资源，析构时释放。依赖“栈对象离开作用域必定析构”（包括异常栈展开）。
- 例子：`std::lock_guard`、`unique_ptr`、`fstream`。
- 追问：析构函数为什么不能抛异常？——栈展开期间再抛会 `std::terminate`；C++11 起析构默认 `noexcept`。

2. shared_ptr 与 unique_ptr 怎么选？★★★

	unique_ptr	shared_ptr
所有权	独占，只能 move	共享，引用计数
大小	一个指针（无状态删除器时）	两个指针（对象指针 + 控制块指针）
开销	零开销	控制块分配 + 原子增减计数

- 默认用 `unique_ptr`，确实需要多方共同持有生命周期时才用 `shared_ptr`。`unique_ptr` 可隐式转成 `shared_ptr`，反之不行。
- 优先 `make_shared`：一次分配对象+控制块，异常安全；缺点是有 weak_ptr 存活时对象内存不能归还。

3. shared_ptr 原理与线程安全 ★★★

- 控制块：强引用计数 use_count、弱引用计数 weak_count、删除器、分配器。
- 强计数归零 → 析构对象；弱计数也归零 → 释放控制块。
- 线程安全：**引用计数的增减是原子的**；但**指向的对象**以及**同一个 shared_ptr 实例本身的读写**不是线程安全的（需加锁或 C++20 `std::atomic<std::shared_ptr>`）。
- 常考手撕：简化版 shared_ptr（拷贝构造/拷贝赋值/移动/析构/自赋值处理）。

4. 循环引用怎么解决？★★★

- A 持有 `shared_ptr<B>`，B 持有 `shared_ptr<A>` → 计数永不归零，泄漏。
- 解决：一方改 `weak_ptr`（通常“子指向父”“观察者”用 weak）。使用时 `lock()` 得到 shared_ptr，判空。
- 追问：`enable_shared_from_this` 作用？——在成员函数内安全拿到自身 shared_ptr，直接 `shared_ptr<T>(this)` 会产生两个控制块导致 double free。

5. 虚函数 / 多态实现 ★★★

- 每个含虚函数的类有一张虚表 vtable（编译期生成，放只读数据段）；每个对象头部有虚表指针 vptr（构造函数中设置）。
- 调用 `p->f()`：取 vptr → 按偏移找表项 → 间接调用。
- 常见追问：
  - 构造函数能是虚函数吗？不能，构造时 vptr 还未就绪。构造/析构中调虚函数不会多态到派生类。
  - 析构函数为什么要虚？基类指针 delete 派生对象，否则只调基类析构 → 泄漏/UB。
  - 纯虚函数、抽象类；`override`/`final` 作用。
  - 多继承下有多个 vptr；菱形继承用虚继承解决重复基类。
  - 静态多态（模板/重载）vs 动态多态（虚函数）。

6. 左值/右值、移动语义 ★★★

- 左值：有名字、可取地址；右值：临时量、字面量。T&& 绑定右值。
- 移动构造：“偷”资源（指针置空），避免深拷贝。移动构造应标 `noexcept`，否则 `vector` 扩容时会退化为拷贝（强异常保证）。
- **std::move 做了什么？**——什么都不“移动”，只是 `static_cast<T&&>`，把表达式转为将亡值，使重载决议选中移动版本。被 move 后的对象处于“有效但未指定”状态。
- `std::forward<T>`：完美转发，配合万能引用 T&& + 引用折叠，保留实参原本的左/右值属性。
- 追问：RVO/NRVO；`return std::move(local)` 反而会妨碍 NRVO。

7. 指针 vs 引用 ★★★

	指针	引用
初始化	可以先不初始化	定义时必须初始化
空值	可以是 `nullptr`	没有“空引用”，必须绑定一个已有对象
改指向	可以随时改指向别的对象	绑定后不能换
`sizeof`	指针本身的大小（64 位下是 8）	被引用对象的大小
多级 / 运算	有 `int**`，能做 p++ 这类运算	没有“引用的引用”、引用数组，也不能做运算

- 底层实现：编译器一般把引用实现成“自动解引用、不能改指向”的指针，所以引用可能也占内存（比如作为类成员时）。
- 怎么选：参数一定有值、而且不需要改指向时用引用（常见写法是 `const T&`，避免拷贝）；可能为空或者需要改指向时用指针。

8. new/delete vs malloc/free ★★★

	new/delete	malloc/free
是什么	C++ 运算符	C 库函数
构造/析构	会调用构造函数和析构函数	不调用，只分配和释放原始内存
大小	编译器按类型自动计算	自己传字节数
返回类型	`T*`，类型安全	`void*`，需要强转
分配失败	抛 `std::bad_alloc`（`new(std::nothrow)` 版本返回 nullptr）	返回 `NULL`
能否重载	能重载 `operator new` / `operator delete`	不能
数组	用 `new[]` 分配，必须用 `delete[]` 释放	没有区分
扩容	没有对应操作	有 `realloc`

- `new T` 实际分两步：先调 `operator new` 分配内存（通常内部就是 malloc），再在这块内存上调用构造函数。
- `delete p` 是反过来：先析构，再调 `operator delete` 释放内存。
- 不能混用：`new` 出来的用 `free` 释放、`new[]` 出来的用 `delete` 释放，都是未定义行为。

9. placement new ★★

在一块已经分配好的内存上构造对象，它本身不分配内存。

```
alignas(T) unsigned char buf[sizeof(T)];
T* p = new (buf) T(args);   // 在 buf 上构造对象
p->~T();                    // 必须手动调用析构，不能 delete p
```

- 用在哪里：内存池、`std::vector` 的实现（先 reserve 原始内存，push_back 时再在上面构造）、`std::optional` 的实现。
- 注意两点：内存必须满足 T 的对齐要求；内存由谁释放，要和这块内存原本的来源保持一致。

10. static、const、volatile、extern、inline 的作用 ★★★

static - 修饰函数内的局部变量：生命周期变成整个程序运行期，只初始化一次。C++11 起这个初始化是线程安全的，所以常用来写单例。 - 修饰全局变量或函数：变成内部链接，只在当前 .cpp 文件里可见。 - 修饰类的静态成员变量：属于类而不属于某个对象，所有对象共享，需要在类外定义（C++17 起可以写成 `inline static` 在类内定义）。 - 修饰类的静态成员函数：没有 `this` 指针，只能访问静态成员，不能是 virtual，也不能是 const。

const - `const int* p` / `int const* p`：p 指向的值不能改，p 本身可以改指向。 - `int* const p`：p 本身不能改指向，指向的值可以改。 - `const T&` 可以绑定临时对象，常用作函数参数，避免拷贝。 - C++ 中 const 全局变量默认是内部链接。 - 如果要求值在编译期就确定，用 `constexpr`。

volatile - 告诉编译器：每次读写都必须真正访问内存，不要把它缓存在寄存器里，也不要把读写优化掉。 - 用在：内存映射的硬件寄存器（MMIO）、信号处理函数里修改的变量（配合 `sig_atomic_t`）、`setjmp/longjmp` 场景。 - **它不保证原子性，也不保证内存顺序，不能用来做多线程同步**，多线程要用 `std::atomic`。

extern - `extern int g;` 只是声明，表示这个变量定义在别的文件里（外部链接）。 - `extern "C"` 关闭 C++ 的名字修饰（name mangling），让 C 和 C++ 代码能互相调用。

inline - 表面含义是建议编译器把函数调用就地展开，但编译器可以忽略这个建议（虚函数通过指针调用、递归函数通常不会被展开）。 - 更关键的作用是允许同一个函数在多个 .cpp 文件里都有定义而不报重复定义（ODR 豁免），所以函数可以直接定义在头文件里。 - 在类体内定义的成员函数默认就是 inline。C++17 起变量也可以声明为 inline。

11. const 成员函数 ★★

```
int size() const;   // 函数内的 this 类型是 const T*
```

- 函数内不能修改成员变量，被 `mutable` 修饰的成员除外（典型用途：缓存、加在 const 函数里的互斥锁）。
- 函数内只能调用其他 const 成员函数。
- const 对象和 const 引用只能调用 const 成员函数。
- 可以按是否 const 进行重载，例如 `T& operator[](size_t)` 和 `const T& operator[](size_t) const` 同时存在。

12. 内存分区 ★★★

地址从高到低：

```
内核空间
栈          局部变量、函数参数、返回地址，向下增长，默认大小约 8MB
mmap 区     共享库、mmap 映射、大块 malloc
堆          new/malloc 分配，向上增长
.bss        未初始化或初始化为 0 的全局/静态变量，在可执行文件里不占空间
.data       已初始化且非 0 的全局/静态变量
.rodata     字符串字面量、const 全局常量
.text       代码段（机器指令）
```

字符串常考的两种写法：

```
const char* p = "hello";  // p 本身在栈上，"hello" 在 .rodata；通过 p 修改会出错（段错误）
char s[] = "hello";       // 把 "hello" 拷贝一份到栈上的数组，可以修改
```

13. 内存对齐 ★★★

为什么要对齐 - CPU 按字长、按缓存行读取内存。数据没对齐时可能要读两次再拼起来，变慢。 - 有些架构（比如部分 ARM）访问未对齐的数据会直接触发异常。 - 原子操作要求操作数是对齐的。

规则 1. 每个成员的起始偏移，必须是它对齐值的整数倍。对齐值取“成员自身大小”和 `#pragma pack` 指定值中较小的那个。 2. 结构体的总大小，必须是所有成员中最大对齐值的整数倍。 3. 结构体嵌套时，内层结构体按它自己的最大对齐值对齐。

```
struct A { char a; int b; char c; };  // 1 + 3(填充) + 4 + 1 + 3(填充) = 12
struct B { int b; char a; char c; };  // 4 + 1 + 1 + 2(填充)           = 8
```

所以按成员大小从大到小排列可以省空间。相关工具：`alignof` 查询对齐值，`alignas` 指定对齐，`#pragma pack(1)` 取消填充（常用于网络协议的报文结构体）。

空类的 sizeof 为什么是 1 - 标准要求每个对象有唯一地址。如果大小是 0，两个空类对象可能落在同一地址，无法区分。 - 空类作为基类时，编译器做“空基类优化”（EBO），它在派生类里不占空间。C++20 起成员变量也可以用 `[[no_unique_address]]` 达到同样效果。 - 只要类里有虚函数，就多一个虚表指针，64 位下 sizeof 至少是 8。

14. 深拷贝/浅拷贝，三/五/零法则 ★★★

浅拷贝：编译器默认生成的拷贝构造是逐个成员拷贝，指针成员只拷贝地址。两个对象指向同一块内存，析构时会重复释放（double free），或者一方释放后另一方变成悬空指针。

深拷贝：拷贝时重新申请一块内存，把内容复制过去。

```
class Buf {
    char* data_; size_t n_;
public:
    Buf(const Buf& o) : data_(new char[o.n_]), n_(o.n_) { memcpy(data_, o.data_, n_); }
    Buf& operator=(Buf o) noexcept { swap(o); return *this; }  // copy-and-swap：自赋值安全、异常安全
    Buf(Buf&& o) noexcept : data_(std::exchange(o.data_, nullptr)), n_(o.n_) {}
    ~Buf() { delete[] data_; }
    void swap(Buf& o) noexcept { std::swap(data_, o.data_); std::swap(n_, o.n_); }
};
```

这里的赋值运算符参数是按值传入，所以同时承担拷贝赋值和移动赋值。

- **三法则**：析构函数、拷贝构造、拷贝赋值，只要需要自定义其中一个，通常三个都要自定义（说明类自己管理着资源）。
- **五法则**：在三法则基础上再加移动构造和移动赋值。注意：只要你自定义了析构函数或拷贝操作，编译器就不再自动生成移动操作，原本该移动的地方会退化成拷贝，性能变差。
- **零法则**：最推荐的做法。成员都用 RAII 类型（`vector`、`string`、`unique_ptr`），这五个函数一个都不写，全部交给编译器生成。

15. lambda 的底层实现 ★★

```
int x = 1, y = 2;
auto f = [x, &y](int a) { return a + x + y; };
```

编译器大致生成下面这样的代码：

```
class __lambda_1 {
    int x;    // 按值捕获：创建 lambda 时拷贝一份，成为成员变量
    int& y;   // 按引用捕获：成员变量是引用
public:
    __lambda_1(int x, int& y) : x(x), y(y) {}
    int operator()(int a) const { return a + x + y; }  // 默认是 const 成员函数
};
auto f = __lambda_1{x, y};
```

常见追问：- `operator()` 默认是 const，所以不能修改按值捕获的变量；加上 `mutable` 才能改。- 按引用捕获时，lambda 活得比被捕获的变量久（比如被异步回调延后执行）就会悬空，这类 bug 很常见。- 没有捕获任何变量的 lambda 可以隐式转换成普通函数指针。- 每个 lambda 都是一个独一无二的类型，保存时用 `auto`、模板参数或 `std::function`。`std::function` 靠类型擦除实现，可能有堆分配开销。- `[this]` 捕获的是 this 指针，对象析构后再调用 lambda 同样会悬空。- C++14 起支持初始化捕获，例如 `[p = std::move(ptr)]`，可以把 `unique_ptr` 移进 lambda。- 泛型 lambda（参数写成 `auto`）的 `operator()` 实际上是一个模板函数。

16. 其他 ★★

- C++11/14/17/20 新特性列举。

二、STL

1. vector 底层与扩容 ★★★

- 三个指针：begin、end、end_of_storage。扩容通常 1.5 倍（MSVC）或 2 倍（GCC），申请新内存 → 移动/拷贝元素 → 释放旧内存。均摊 O(1) push_back。
- `reserve` 只改容量，`resize` 改 size 并构造元素。`shrink_to_fit` / swap 技巧释放内存。

2. 迭代器失效 ★★★

容器	插入	删除
vector	触发扩容：**全部失效**；未扩容：插入点及之后失效	删除点及之后失效
deque	首尾插入：迭代器全失效，引用不失效；中间插入：全失效	首尾删除只影响被删元素；中间删除全失效
list / map / set	不失效	仅被删元素失效
unordered_*	rehash 时迭代器全失效（引用/指针不失效）	仅被删元素失效

- 正确的循环删除写法：`it = v.erase(it);`（map 同理，C++11 起 erase 返回下一个）。

3. map vs unordered_map ★★★

- map：红黑树，有序，O(log n)，迭代器稳定，key 需要 `operator<`。
- unordered_map：哈希表（桶数组 + 链表，拉链法），平均 O(1)，最坏 O(n)，key 需要 hash 与 ==。
- 什么时候选有序容器：需要有序遍历、范围查询（`lower_bound`）、求前驱后继、数据量小、对最坏延迟敏感、key 难以写好 hash。
- 红黑树性质（5 条）与为何不用 AVL：插删旋转更少，适合频繁修改。

4. 哈希冲突怎么处理？★★★

- 开放定址（线性/二次探测、双重哈希）、拉链法（STL 采用）、再哈希、公共溢出区。
- 负载因子 `size/bucket_count` 超过 `max_load_factor`（默认 1.0）时 rehash，桶数一般扩到下一个质数。
- 追问：自定义类型做 key 需要提供 hash 函数与相等比较。

5. 其他 ★★

- deque 结构（中控器 map + 多个定长缓冲区）。
- list vs vector 选择（缓存局部性通常让 vector 更快）。
- `priority_queue` 底层是堆（vector + make_heap）。
- `emplace_back` vs `push_back`。
- STL 容器线程安全吗？——不是，多线程读安全，有写需外部同步。
- 手撕：LRU（`list` + `unordered_map`）。

三、操作系统

1. 进程 vs 线程 ★★★

- 进程是资源分配单位，线程是调度单位。Linux 中二者都是 `task_struct`，线程由 `clone` 共享资源创建。
- **线程共享**：地址空间（代码段、数据段、堆）、打开的文件描述符、信号处理方式、当前工作目录、用户/组 ID。
- **线程私有**：栈、寄存器（含 PC）、线程 ID、errno、信号屏蔽字、调度优先级、TLS（`thread_local`）。
- 切换成本：进程切换需换页表（CR3）导致 TLB 失效，线程切换不用。
- 追问：协程与线程区别；全局变量又想线程独有 → `thread_local`。

2. 进程间通信 IPC ★★★

管道/命名管道、消息队列、共享内存（最快，需配合同步）、信号量、信号、socket（含 Unix domain socket）、mmap。

3. 虚拟内存 ★★★

- 为什么需要：进程隔离、每个进程有连续独立的地址空间、按需分配/换出、共享库与写时复制。
- 进程地址空间布局（低→高）：代码段、数据段、BSS、堆（向上）、mmap 区、栈（向下）、内核空间。

4. 一次内存访问如何完成地址转换？ ★★★

1. CPU 发出虚拟地址，MMU 先查 **TLB**，命中直接得到物理页号。
2. 未命中则做**页表遍历**：x86-64 四级页表（PGD→PUD→PMD→PTE），CR3 存顶级页表物理地址，虚拟地址按 9/9/9/9/12 位切分索引各级。
3. 找到 PTE，检查存在位与权限位；合法则物理页号 + 页内偏移 = 物理地址，并回填 TLB。
4. 用物理地址访问 Cache → 主存。
5. PTE 不存在或权限不符 → **缺页异常**。

- 追问：为什么多级页表？——稀疏地址空间下节省页表内存。大页（HugePage）减少 TLB miss。

5. 缺页异常 ★★

- 次要缺页（minor）：页在内存中（如 page cache、首次访问匿名页），只需建立映射。
- 主要缺页（major）：需从磁盘/swap 读入。
- 非法访问 → `SIGSEGV`。
- 写时复制（COW）：fork 后父子共享只读页，写时触发缺页再复制。
- 页面置换算法：FIFO、LRU、Clock、LFU；Linux 用 active/inactive 双链表近似 LRU。

6. 堆与栈 ★★

- 栈：编译器自动管理，移动栈指针分配，大小有限（默认常 8MB，`ulimit -s`），过深递归栈溢出。
- 堆：`malloc`，glibc ptmalloc 小于 128KB 走 `brk`，大块走 `mmap`；有碎片问题。tcmalloc/jemalloc 用线程缓存减少锁竞争。
- 函数调用过程：压参数/返回地址 → 保存 rbp → 分配栈帧 → 返回时恢复。

7. 其他 ★★

- 用户态 vs 内核态，系统调用过程。
- 进程状态（R/S/D/Z/T），僵尸进程与孤儿进程，如何避免僵尸（wait/SIGCHLD）。
- fork / exec / COW；守护进程创建步骤。
- 调度算法（CFS 红黑树按 vruntime）。
- 软链接 vs 硬链接；page cache；`free` 中 buffer 与 cache；mmap 原理与零拷贝（sendfile）。

四、并发编程

1. 互斥锁 vs 自旋锁 ★★★

- 互斥锁：拿不到锁则阻塞睡眠，让出 CPU，有上下文切换开销。Linux 下基于 **futex**：无竞争时纯用户态原子操作，有竞争才陷入内核。
- 自旋锁：拿不到锁就忙等，不切换上下文；适合临界区极短、多核、不可睡眠的场景（内核中断上下文）。单核上自旋无意义。
- 读写锁：读多写少；注意写者饥饿。
- 追问：mutex 底层原理（原子 CAS + futex 等待队列）。

2. 条件变量为什么要配合循环检查？★★★

- **虚假唤醒**：`pthread_cond_wait` 规范允许无信号返回。
- **被抢先**：被唤醒到重新拿到锁之间，条件可能已被其它线程改变（如 `notify_all` 时多个消费者竞争同一个元素）。

- 因此写成 `while (!cond) cv.wait(lk);` 或 `cv.wait(lk, pred);`。
- 为什么 wait 要传入锁：检查条件与进入等待必须原子化，否则会丢失唤醒（lost wakeup）。

3. 生产者消费者怎么写？★★★

```
template <class T>
class BlockingQueue {
    std::mutex mu_;
    std::condition_variable not_empty_, not_full_;
    std::queue<T> q_;
    size_t cap_;
public:
    explicit BlockingQueue(size_t cap) : cap_(cap) {}
    void push(T v) {
        std::unique_lock<std::mutex> lk(mu_);
        not_full_.wait(lk, [&] { return q_.size() < cap_; });
        q_.push(std::move(v));
        not_empty_.notify_one();
    }
    T pop() {
        std::unique_lock<std::mutex> lk(mu_);
        not_empty_.wait(lk, [&] { return !q_.empty(); });
        T v = std::move(q_.front());
        q_.pop();
        not_full_.notify_one();
        return v;
    }
};
```

要点：两个条件变量、谓词循环、锁内修改共享状态。

4. 如何设计线程池？★★★（常手撕）

- 组成：任务队列（上面的阻塞队列）+ 固定数量工作线程 + 停止标志。
- 工作线程循环：等待任务 → 取出 → 锁外执行。
- 提交接口：`submit(F&&, Args&&...)` 用 `std::packaged_task` 包装，返回 `std::future`。
- 关闭：置 `stop=true` → `notify_all` → 工作线程把队列剩余任务处理完后退出 → `join` 所有线程。
- 扩展点：线程数选择（CPU 密集 ≈ 核数，I/O 密集可更多）、有界队列与拒绝策略、任务异常处理、动态伸缩、工作窃取。

5. 原子操作与内存序 ★★

- `std::atomic`：硬件 `lock` 前缀 / LL-SC 实现，保证读改写不可分割。
- CAS：`compare_exchange_weak/strong`，无锁结构的基础；ABA 问题（用版本号/tagged pointer 解决）。

- 内存序：`relaxed`（只保证原子性）、`acquire/release`（建立 happens-before，常用于发布数据）、`seq_cst`（默认，全局一致顺序）。
- `volatile` 不能保证线程安全。

6. 死锁 ★★★

- 四个必要条件：互斥、占有并等待、不可剥夺、循环等待。
- 预防：固定加锁顺序（破坏循环等待）、`std::lock`/`std::scoped_lock` 一次性拿多把锁、`try_lock` + 超时回退、减小锁粒度。
- 排查：`gdb attach` → `thread apply all bt` 查看各线程卡在哪把锁；`pstack`；valgrind helgrind / TSan。

7. 其他 ★★

- 线程安全的单例（C++11 局部静态变量即线程安全；DCLP 需原子）。
- 无锁队列思路；`shared_ptr` 线程安全性；`std::async`/`future`/`promise`。
- 线程同步方式汇总：mutex、recursive_mutex、shared_mutex、condition_variable、信号量（C++20 `counting_semaphore`）、屏障、原子变量。

五、计算机网络

1. TCP 三次握手 ★★★

- SYN(seq=x) → SYN+ACK(seq=y, ack=x+1) → ACK(ack=y+1)。状态：客户端 SYN_SENT → ESTABLISHED；服务端 LISTEN → SYN_RCVD → ESTABLISHED。
- 为什么三次：确认双方收发能力并同步双方初始序列号；防止历史重复 SYN 建立错误连接。两次不够，四次多余。
- 追问：半连接队列 / 全连接队列（`backlog`、`somaxconn`）；SYN Flood 与 SYN Cookie；第三次 ACK 丢失会怎样。

2. 四次挥手 ★★★

- FIN → ACK → FIN → ACK。主动方：FIN_WAIT_1 → FIN_WAIT_2 → TIME_WAIT → CLOSED；被动方：CLOSE_WAIT → LAST_ACK → CLOSED。
- 为什么四次：TCP 全双工，被动方收到 FIN 时可能还有数据要发，ACK 与 FIN 分开（无数据时可合并成三次）。
- 大量 CLOSE_WAIT：说明**应用程序没有调用 close**，是代码 bug。

3. 为什么需要 TIME_WAIT？ ★★★

- **可靠终止**：最后的 ACK 若丢失，对端重发 FIN，主动方还能重发 ACK；否则对端会收到 RST。

- **让旧连接的残留报文在网络中消失**：等待 2MSL，避免相同四元组的新连接收到旧数据。
- Linux 中 TIME_WAIT 固定 60s。
- 大量 TIME_WAIT 怎么办：使用长连接/连接池；`SO_REUSEADDR`（服务端重启绑定端口）；`net.ipv4.tcp_tw_reuse`（客户端主动连接复用）；扩大本地端口范围。`tcp_tw_recycle` 已在 4.12 移除（NAT 下有问题）。

4. 可靠传输 ★★★

序列号与确认号、超时重传（RTO 依据 RTT 动态计算）、快速重传（3 个重复 ACK）、SACK、滑动窗口、校验和、去重排序。

5. 流量控制 vs 拥塞控制 ★★★

- 流量控制：保护**接收方**，接收方通过 rwnd 通告剩余缓冲；零窗口时发送方启动持续定时器探测。
- 拥塞控制：保护**网络**，发送方维护 cwnd。
  - 慢启动（指数增长至 ssthresh）→ 拥塞避免（线性增长）→ 超时：ssthresh=cwnd/2，cwnd=1 重新慢启动 → 3 个重复 ACK：快重传 + 快恢复（cwnd=ssthresh+3）。
  - 现代算法：CUBIC（Linux 默认）、BBR（基于带宽与 RTT 估计）。
- 实际发送窗口 = min(rwnd, cwnd)。

6. 应用层如何处理消息边界（粘包/拆包）？ ★★★

- 原因：TCP 是字节流，没有消息边界；Nagle 算法、发送/接收缓冲、MSS 分段都会导致合并或拆分。
- 解决：
  1. 定长消息；
  2. 分隔符（如 HTTP 头用 `\r\n\r\n`、Redis RESP）；
  3. **长度前缀（最常用）**：固定头部含 body 长度，接收方用缓冲区累积，够一个头再按长度取完整包，不足继续等待。
- 实现要点：每个连接维护应用层接收缓冲；网络字节序（`htonl`）；校验长度上限防恶意包。

7. 其他 ★★

- TCP vs UDP；UDP 如何实现可靠（QUIC/KCP）。
- 输入 URL 到页面展示全过程；DNS 解析；HTTP/1.1、HTTP/2、HTTP/3 区别；HTTPS（TLS 握手）。
- Nagle 与延迟确认、`TCP_NODELAY`；keepalive；MSS/MTU。
- `ping` 原理（ICMP，网络层）；大小端转换。
- 连接断开的判定方式（read 返回 0、RST/EPIPE、心跳超时）。

六、Linux I/O

1. 阻塞/非阻塞、同步/异步 ★★★

- 阻塞：数据没准备好时调用线程挂起；非阻塞：立即返回 EAGAIN。
- 同步 I/O：数据从内核拷贝到用户空间这一步由调用者等待（阻塞、非阻塞、多路复用、信号驱动都是同步）。异步 I/O：内核完成拷贝后再通知（io_uring、POSIX AIO）。
- 五种 I/O 模型：阻塞、非阻塞、I/O 多路复用、信号驱动、异步。

2. select、poll、epoll 的区别 ★★★

	select	poll	epoll
fd 上限	FD_SETSIZE 默认 1024	无（链表/数组）	无（受系统 fd 上限）
数据结构	位图	pollfd 数组	内核红黑树（存监听 fd）+ 就绪链表
每次调用拷贝	全量 fd 集合拷入内核	全量拷贝	注册一次（epoll_ctl），wait 只返回就绪 fd
检测就绪	内核遍历 O(n)，用户也要遍历	同左	回调把就绪 fd 挂到就绪链表，O(就绪数)
触发方式	LT	LT	LT / ET

- 结论：连接多、活跃少时 epoll 优势明显；连接少且都活跃时差距不大。
- 追问：epoll 用 mmap 共享内存？——这是常见误传，实际 `epoll_wait` 通过 `copy_to_user` 拷贝就绪事件。

3. LT / ET 怎么用？★★★

- LT（水平触发，默认）：只要缓冲区还有数据/可写，每次 `epoll_wait` 都会通知。编程简单，可以不一次读完。
- ET（边缘触发）：只在状态变化时通知一次。
  - **必须配合非阻塞 fd**；
  - **必须循环 read/accept 直到返回 `EAGAIN`**，否则剩余数据不会再通知；
  - 减少重复通知次数，效率更高，但编程易出错。
- 写事件：LT 下不要一直注册 EPOLLOUT（会忙轮询），只在发送缓冲写满时注册，写完后取消。
- `EPOLLONESHOT`：多线程下保证一个 fd 同一时刻只被一个线程处理，处理完需重新 `epoll_ctl` 激活。
- 惊群：`accept` 惊群内核已解决；epoll 惊群用 `EPOLLEXCLUSIVE` 或 `SO_REUSEPORT`。

4. 服务器模型 ★★★

- Reactor：同步 I/O 多路复用 + 事件分发。单 Reactor 单线程（Redis 6 前）、单 Reactor 多线程、**主从 Reactor**（主线程 accept，子线程各自一个 epoll 处理连接，如 muduo、Netty、Nginx 多进程类似）。
- Proactor：基于异步 I/O，内核完成读写后通知（Windows IOCP、io_uring）。
- one loop per thread + 线程池处理计算任务，是 C++ 网络库常见架构。

5. 其他 ★★

- 零拷贝：`mmap + write`、`sendfile`、`splice`；传统 read+write 4 次拷贝 4 次切换。
- `fflush` vs `fsync`；page cache 与 Direct I/O。
- io_uring 基本原理（提交队列 SQ、完成队列 CQ 共享环形缓冲）。

七、排障与性能

1. 进程 CPU 高怎么定位？★★★

1. `top` 找到进程 → `top -Hp <pid>` 找到高 CPU 的线程 TID。
2. 区分 us/sy/wa/si：us 高 → 用户代码；sy 高 → 系统调用/锁/上下文切换多；wa 高 → I/O 等待。
3. 采样定位热点：`perf top -p <pid>`、`perf record -g -p <pid>` + `perf report`，或生成**火焰图**。
4. 快照：`pstack <pid>` / `gdb attach` + `thread apply all bt` 多抓几次，看哪段栈反复出现（死循环、自旋）。
5. 常见原因：死循环、忙轮询（如 EPOLLOUT 一直注册）、频繁加锁/自旋、正则/序列化热点、GC/内存分配过多。

2. 内存持续增长怎么查？★★★

1. 先确认：`top`/ps 看 RSS/VIRT 趋势；`/proc/<pid>/status`、`/proc/<pid>/smaps`、`pmap -x` 看增长在堆、mmap 还是栈。
2. 区分泄漏 vs 缓存/碎片：
    - 真泄漏：Valgrind memcheck（测试环境）、AddressSanitizer（`-fsanitize=address`，LeakSanitizer）、heaptrack；
    - 线上：tcmalloc/jemalloc 的 heap profiler、`perf`/eBPF `memleak`（bcc）跟踪未释放分配；
    - 不是泄漏但不还给系统：glibc 分配器碎片、多 arena（可试 `malloc_trim`、`MALLOC_ARENA_MAX`、换 jemalloc）。
3. 代码层常见原因：容器只增不删（无上限缓存、map 未清理）、shared_ptr 循环引用、未匹配的 new/delete、连接/任务对象堆积（下游慢导致队列无限增长）。

3. 吞吐下降从哪里看？★★★

按 USE 方法（Utilization 使用率 / Saturation 饱和度 / Errors 错误）逐个资源排查： - **CPU**：`top`、`mpstat -P ALL`（是否单核打满）、`vmstat` 看 r 队列和 cs（上下文切换）。 - **内存**：`free`、`vmstat` 看 si/so（是否在 swap）、缺页数 `pidstat -r`。 - **磁盘 I/O**：`iostat -x` 看 %util、await、队列长度；`iotop`、`pidstat -d`。 - **网络**：`sar -n DEV` 带宽、`ss -s`/`ss -ant` 连接状态（TIME_WAIT/CLOSE_WAIT 堆积）、`netstat -s` 重传与丢包、全连接队列溢出（`ListenOverflows`）。 - **锁竞争**：`pidstat -w` 看自愿/非自愿上下文切换；`perf lock`、`perf record` 中 futex/`pthread_mutex_lock` 占比高；off-CPU 火焰图。 - **应用层**：线程池队列长度、请求延迟分位数（P99）、下游依赖（DB/RPC）耗时、日志同步写盘、GC/分配器。 - 对比基线：上线变更、流量变化、依赖变化，先定位“何时开始变差”。

4. 锁竞争怎么优化？★★

减小临界区、拆分锁（分段锁/按 key 分片）、读写锁、无锁结构/原子变量、thread_local 减少共享、批量处理、避免锁内做 I/O。

5. 常用工具清单 ★★

场景	工具
概览	top/htop, vmstat, dstat, uptime（load）
CPU 热点	perf, 火焰图, gprof
栈快照	gdb, pstack, gcore
内存	valgrind, ASan/LSan, heaptrack, pmap, /proc/pid/smaps
I/O	iostat, iotop, pidstat -d, strace
网络	ss, netstat, tcpdump, wireshark, sar -n
系统调用	strace -c -p, ltrace
动态追踪	eBPF / bcc / bpftrace
崩溃	core dump（`ulimit -c unlimited`）+ gdb bt

八、复习优先级建议

1. **第一梯队（必须能脱稿讲 3 分钟）**：智能指针与循环引用、虚函数表、移动语义、vector 扩容与迭代器失效、unordered_map 冲突与 rehash、进程线程区别、虚拟地址转换、条件变量 + 生产者消费者、TCP 握手挥手 + TIME_WAIT、粘包处理、select/poll/epoll + LT/ET、CPU 高与内存泄漏排查流程。
2. **第二梯队（能说要点）**：缺页与 COW、内存序、死锁预防与排查、拥塞控制细节、Reactor 模型、零拷贝、吞吐下降排查。
3. **手撕高频**：线程池、shared_ptr、阻塞队列、LRU、单例、memcpy（考虑重叠）。

4. **结合项目**：每个考点准备一句“我在项目里遇到/用过……”，面试官最爱从八股追到项目。